

python - numpy explanation

NumPy Fundamentals — Quiz Prep Notes

Source: analysis of `numpy-sample.ipynb`

Goal: understand every line well enough to answer quiz questions about it, not just recognize it.

0. The Big Picture — Why does NumPy even exist?

Before the code, one idea has to click: **a Python `list` and a NumPy `array` are NOT the same thing**, even though they can look similar.

Feature	Python <code>list</code>	NumPy <code>ndarray</code>
Can hold mixed types (<code>[1, "a", 3.5]</code>)?	Yes	No — all elements share one data type
Speed on large data	Slow (pure Python loop)	Fast (runs in optimized C code)
Math like <code>arr * 2</code>	Multiplies the <i>list length</i> (repeats it)	Multiplies every element by 2
Built for	General storage	Numerical / scientific computing

Quiz angle

If a question asks "why use NumPy instead of a list?" — the answer is almost always **speed + true element-wise math**, because NumPy arrays are stored in contiguous memory as a single data type, unlike Python lists which store pointers to separate objects.

1. Importing NumPy

```
1 import numpy as np
```

- `numpy` is a third-party library — it must be installed (`pip install numpy`) and imported before use.

- `as np` is not required syntax, it's a **universal convention**. Everyone in the numeric-Python world writes `np.something` instead of `numpy.something`. A quiz might trip you up by importing it as something else and still expecting you to know it's NumPy.

2. Creating an Array from a List — `np.array()`

```
1 age_list = [25, 30, 35, 40, 45]
2 age_array = np.array(age_list)
3 print(type(age_array))
4 print(age_array)
```

Output:

```
1 <class 'numpy.ndarray'>
2 [25 30 35 40 45]
```

What's happening

- `np.array()` is the **constructor function** that converts a normal Python list (or tuple) into a NumPy array object.
- The resulting object's type is `numpy.ndarray` ("n-dimensional array") — this is the **core data structure of the entire NumPy library**. Every array you ever create in NumPy, 1D or 100D, is an `ndarray`.
- Notice the printed array has **no commas** between numbers (`[25 30 35 40 45]`) — that's a dead giveaway it's a NumPy array and not a Python list (which would print `[25, 30, 35, 40, 45]`).

Common quiz trap

`type(age_array)` returns `numpy.ndarray`, **not** `list` and **not** `array` alone. Memorize the exact class name.

3. Inspecting an Array — `.shape`, `.ndim`, `.dtype`, `.size`

```
1 print('Shape: ', age_array.shape)
```

```

2 print('Dimensions: ', age_array.ndim)
3 print('Data type: ', age_array.dtype)
4 print('Size: ', age_array.size)

```

Output:

```

1 Shape: (5,)
2 Dimensions: 1
3 Data type: int64
4 Size: 5

```

These four are **attributes**, not methods — notice there are **no parentheses** `()` after them. This is a classic thing quizzes test: writing `.shape()` instead of `.shape` is wrong and will error.

Attribute	Meaning	In this example
<code>.shape</code>	A tuple telling you the size along each axis/dimension	<code>(5,)</code> → one axis, with 5 elements. The trailing comma matters — it signals "this is a 1-element tuple," not just the number 5.
<code>.ndim</code>	How many dimensions (axes) the array has	<code>1</code> → it's a flat, 1D array (like a single row/column)
<code>.dtype</code>	The single data type shared by every element	<code>int64</code> → 64-bit integers
<code>.size</code>	Total number of elements (product of all shape values)	<code>5</code>

Relationship to remember

`size` = the product of everything inside `shape` .

For shape `(5,)` , `size` = 5. Later for shape `(3,3)` , `size` = $3 \times 3 = 9$.

Quizzes love asking you to compute `.size` from `.shape` or vice versa.

4. Special Array Generators — `zeros` , `ones` , `arange`

```

1 a = np.zeros(5)
2 b = np.ones(5)

```

```
3 c = np.arange(0, 20, 2)
4 print(a)
5 print(b)
6 print(c)
```

Output:

```
1 [0. 0. 0. 0. 0.]
2 [1. 1. 1. 1. 1.]
3 [ 0  2  4  6  8 10 12 14 16 18]
```

`np.zeros(n)`

Creates an array of `n` zeros. **Default dtype is** `float64`, which is why the output shows `0.` (with a decimal point) and not plain `0`.

`np.ones(n)`

Same idea, but filled with `1.0` s. Also defaults to floats.

⚠ Common quiz trap

Students expect `np.zeros(5)` to give integers `[0 0 0 0 0]`. It doesn't — by default it gives **floats** `[0. 0. 0. 0. 0.]`.

If you need integers, you must specify it: `np.zeros(5, dtype=int)`.

`np.arange(start, stop, step)`

This is NumPy's version of Python's built-in `range()`, but it returns an array instead of a range object.

- `np.arange(0, 20, 2)` → start at 0, stop **before** 20, step by 2.
- Result: `0, 2, 4, 6, 8, 10, 12, 14, 16, 18` — **10 elements** (note that 20 itself is excluded, same "stop is exclusive" rule as Python's `range`).

🔗 Quiz angle

`arange` is exclusive of the stop value, just like `range()`.

If asked "how many elements does `np.arange(0, 20, 2)` produce," count using

$(\text{stop} - \text{start}) / \text{step} = (20-0)/2 = \mathbf{10}$, not 11.

5. 2D Arrays / Matrices

```
1 matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
2 print(matrix)
3 print('Shape: ', matrix.shape)
4 print('Dimensions: ', matrix.ndim)
5 print('Data type: ', matrix.dtype)
6 print('Size: ', matrix.size)
```

Output:

```
1  [[1 2 3]
2   [4 5 6]
3   [7 8 9]]
4  Shape: (3, 3)
5  Dimensions: 2
6  Data type: int64
7  Size: 9
```

What changed vs. the 1D example

- Instead of passing a flat list to `np.array()`, you pass a **list of lists**. Each inner list becomes one **row**.
- `.shape` is now `(3, 3)` → **3 rows, 3 columns**. This is the key pattern: `shape` for a 2D array is always `(rows, columns)`.
- `.ndim` is `2` because there are two axes to move along: axis 0 (down the rows) and axis 1 (across the columns).
- `.size` is `9` because `3 rows × 3 columns = 9 total elements` — confirming the "size = product of shape" rule from Section 3.

Quiz angle

Given any nested list `[[a,b,c],[d,e,f]]`, the shape is (number of inner lists, length of each inner list) — **as long as every inner list is the same length**. If lengths differ, NumPy can't form a clean rectangular array and will either error or

create a less useful "object" array (in older NumPy) — a good thing to be aware of even if not shown in this notebook.

What was commented out in the notebook

There's a commented-out example in the source that tries to build a **4D array** (a list of lists of lists of lists). It was commented out probably to keep the demo simple, but it hints that `ndim` isn't limited to 1 or 2 — NumPy supports arbitrarily many dimensions. Each extra level of nested brackets `[ ]` adds one more dimension.

6. `np.arange()` for a Bigger 1D Array

```
1 a = np.arange(12)
2 print(a)
```

Output:

```
1 [ 0  1  2  3  4  5  6  7  8  9 10 11]
```

- Called with a **single argument**, `np.arange(12)` assumes `start=0` and `step=1` by default — same shortcut behavior as `range(12)`.
- Produces 12 elements: `0` through `11` (12 excluded, since stop is exclusive — same rule as Section 4).
- This array is created specifically so it can be **reshaped** next — the number 12 isn't random, it's chosen because 12 factors nicely into 3×4 (see below).

7. Reshaping an Array — `.reshape()`

```
1 matrix = a.reshape(3, 4)
2 print(matrix)
```

Output:

```
1  [[ 0  1  2  3]
2   [ 4  5  6  7]
3   [ 8  9 10 11]]
```

What `.reshape()` does

- It takes the **same 12 elements**, in the same order, and rearranges them into a new shape — here, **3 rows × 4 columns**.
- It does **not** change the data or create new values — just changes how the *same* elements are organized/viewed.
- The array is filled **row by row** ("row-major" / C-style order by default): the first 4 numbers (0, 1, 2, 3) become row 1, the next 4 (4, 5, 6, 7) become row 2, and so on.

⚠ The #1 rule of reshape — and the #1 quiz trap

The total number of elements must stay exactly the same before and after reshaping.

`a` has 12 elements (`.size == 12`). We reshaped it into (3, 4), and $3 \times 4 = 12$ ✓. If you tried `a.reshape(3, 5)` ($3 \times 5 = 15 \neq 12$), NumPy would raise a `ValueError` because 15 elements are needed but only 12 exist.

Quick check formula: `new_shape[0] × new_shape[1] × ... == original.size`

Full Concept Map (quick recall)

1	<code>np.array(list)</code>	→ build an ndarray from existing Python data
2	<code>np.zeros(n)</code>	→ array of n zeros (floats by default)
3	<code>np.ones(n)</code>	→ array of n ones (floats by default)
4	<code>np.arange(start, stop, step)</code>	→ like <code>range()</code> , but returns an array
5	<code>.shape</code>	→ tuple describing size per dimension, e.g. (rows, cols)
6	<code>.ndim</code>	→ number of dimensions (axes)
7	<code>.dtype</code>	→ the single shared data type of all elements
8	<code>.size</code>	→ total element count = product of all values in <code>.shape</code>
9	<code>.reshape(new_shape)</code>	→ rearrange same elements into a new shape
10		(must satisfy: new total size == original size)

Self-Check Questions (try answering before your quiz)

1. What is the exact output of `type()` when called on any NumPy array?
2. Why does `np.zeros(4)` print `[0. 0. 0. 0.]` instead of `[0 0 0 0]`?
3. If `arr.shape == (4, 6)`, what is `arr.ndim` and what is `arr.size`?
4. How many elements does `np.arange(5, 30, 5)` produce, and what are they?
5. You have an array with `.size == 20`. Which of these reshape calls will work: `.reshape(4, 5)`, `.reshape(2, 10)`, `.reshape(3, 7)`? Why or why not?
6. What's the difference between a Python `list` and a NumPy `ndarray` in terms of allowed data types?

✓ Answers (click to expand) >

1. `<class 'numpy.ndarray'>`
2. Because `np.zeros()` defaults to `float64` dtype, and floats always print with a decimal point.
3. `ndim = 2` (it's a 2D array); `size = 4 × 6 = 24`.
4. $(30 - 5) / 5 = 5$ elements: 5, 10, 15, 20, 25 (30 itself is excluded).
5. `.reshape(4, 5)` works ($4 \times 5 = 20$ ✓).
`.reshape(2, 10)` works ($2 \times 10 = 20$ ✓).
`.reshape(3, 7)` fails ($3 \times 7 = 21 \neq 20$ ✗) — raises a `ValueError`.
6. A Python list can mix types freely (`[1, "a", 3.5]` is valid).
A NumPy array forces **one shared dtype** for all elements — if you mix types, NumPy will upcast everything to a common type (e.g., all ints become floats if a float is present).